

Project Description – DIY Transmitter and Receiver for a RC Jet

DESCRIPTION

1. What is this ?
 - This is a transmitter receiver model made from scratch , specifically meant for Radio Controlled Vehicles .
 - The brain of the system is composed of Arduino Nano clones and the communication modules are made up of NRF24L01 modules .
 - This project was inspired by a video made by a Youtuber named Electronoob , whose content greatly simplified the process of making the project .
2. Why was this made ?
 - This project was made as an initializer for me into embedded electronics and was mainly a reality check for my own competence .
 - This project was aimed at cutting the costs of buying an off the shelf transmitter receiver set , which was successfully met with this project being completed in one third of the price of a commercial TXRX set.
3. What does this repository contain ?
 - Transmitter firmware and hardware documentation
 - Receiver firmware and signal handling logic
 - Integration notes for a fixed-wing RC jet
 - Debugging notes and testing observations
4. Did the aircraft fly?
 - The airframe was fully assembled and functionally validated through ground and bench testing.
 - A brief flight attempt was conducted; however, full flight testing and video documentation were intentionally limited due to environmental and airspace safety considerations.
 - This repository therefore emphasizes system design and validation over flight footage.
5. Most Challenging part of the project
 - Ensuring reliable communication and predictable actuator behaviour under real-world constraints such as noise, timing issues, and signal loss.

Transmitter Overview

This module implements the transmitter-side firmware for the custom RC link.

It is responsible for reading user input controls and transmitting them as structured control data to the receiver.

Core Functionality

- Read analog input values from control interfaces
- Normalize and map inputs to defined control ranges
- Package control data into a fixed-format packet
- Transmit packets at a consistent update rate

Design Notes

The transmitter logic is intentionally kept simple and deterministic. No adaptive or autonomous behavior is implemented; all outputs directly reflect user inputs after basic conditioning.

Testing Notes

Functionality was validated through:

- Input range checks
 - Continuous transmission observation
 - End-to-end response verification with the receiver
-

Scope

This module is designed for manual control experimentation and educational use.

It prioritizes clarity and predictability over advanced features or optimizations.

Receiver Overview:

This module implements the receiver-side firmware for the custom RC link.

It is responsible for decoding control packets, validating link integrity, and generating safe, predictable actuator outputs.

Core Responsibilities

- Receive and parse RF control packets
 - Map decoded values to PWM signals for servos and ESC
 - Maintain deterministic update timing
 - Enforce safe default states on signal loss (failsafe)
-

Failsafe Behavior

In the absence of valid packets beyond a defined timeout:

- Control outputs are driven to predefined safe values
- Throttle is reduced to a non-driving state
- Actuator outputs remain stable to avoid oscillations

This ensures predictable behavior under link degradation or loss.

Testing Notes

Receiver behavior was validated through bench testing, including:

- Packet loss simulation
 - Power cycling scenarios
-

Scope

This module is intended for experimental and educational use and was developed with an emphasis on robust signal handling and safety-aware design, rather than flight performance.

CODE AND PICTURES:

Transmitter Code:

```
/* Transmitter code for the Arduino Radio control with PWM or PPM output
 * Install the NRF24 Library to your IDE
 * Upload this code to the Arduino UNO, NANO, Pro mini (5V,16MHz)
 * Connect a NRF24 module to it:

    Module // Arduino UNO,NANO

    GND    ->    GND
    Vcc    ->    3.3V
    CE     ->    D9
    CSN    ->    D10
    CLK    ->    D13
    MOSI   ->    D11
    MISO   ->    D12

This code transmits 7 channels with data from pins A0, A1, A2, A3, A4, D2 and D3
Please, like share and subscribe : https://www.youtube.com/c/ELECTRONOOBS
*/

#include <SPI.h>
#include <nRF24L01.h> //Remember to install this library:
http://www.electronoobs.com/engarduino\_nrf24\_lib.php
#include <RF24.h>

const uint64_t my_radio_pipe = 0xE8E8F0F0E1LL; //Remember that this code should be the same for the receiver

RF24 radio(9, 10); //Set CE and CSN pins

// The sizeof this struct should not exceed 32 bytes
struct Data_to_be_sent {
    byte ch1;
    byte ch2;
    byte ch3;
    byte ch4;
    byte ch5;
    byte ch6;
    byte ch7;
};

//Create a variable with the structure above and name it sent_data
Data_to_be_sent sent_data;

void setup()
{
    radio.begin();
    radio.setAutoAck(false);
    radio.setDataRate(RF24_250KBPS);
    radio.openWritingPipe(my_radio_pipe);

    //Reset each channel value
    sent_data.ch1 = 127;
    sent_data.ch2 = 127;
    sent_data.ch3 = 127;
    sent_data.ch4 = 127;
    sent_data.ch5 = 0;
    sent_data.ch6 = 0;
    sent_data.ch7 = 0;
}

/*****/

void loop()
{
    /*If your channel is reversed, just swap 0 to 255 by 255 to 0 below
    EXAMPLE:
```

```

Normal:    data.ch1 = map( analogRead(A0), 0, 1024, 0, 255);
Reversed:  data.ch1 = map( analogRead(A0), 0, 1024, 255, 0);  */

sent_data.ch1 = map( analogRead(A0), 0, 1024, 0, 255);
sent_data.ch2 = map( analogRead(A1), 0, 1024, 0, 255);
sent_data.ch3 = map( analogRead(A2), 0, 1024, 0, 255);
sent_data.ch4 = map( analogRead(A3), 0, 1024, 0, 255);
sent_data.ch5 = digitalRead(2);
sent_data.ch6 = digitalRead(3);
sent_data.ch7 = map( analogRead(A4), 0, 1024, 0, 255);

radio.write(&sent_data, sizeof(Data_to_be_sent));
}

```

Receiver Code:

```

/* Receiver code for the Arduino Radio control with PWM output
 * Install the NRF24 Library to your IDE
 * Upload this code to the Arduino UNO, NANO, Pro mini (5V,16MHz)
 * Connect a NRF24 module to it:

Module // Arduino UNO,NANO

GND    ->   GND
Vcc    ->   3.3V
CE     ->   D9
CSN    ->   D10
CLK    ->   D13
MOSI   ->   D11
MISO   ->   D12

This code receive 7 channels and create a PWM output for each one on D2, D3, D4, D5, D6, D7 and D8
Please, like share and subscribe : https://www.youtube.com/c/ELECTRONO0BS
*/

#include <SPI.h>
#include <nRF24L01.h>
#include <RF24.h>
#include <Servo.h> //To create PWM signals we need this Lybrary

const uint64_t pipeIn = 0xE8E8F0F0E1LL; //Remember that this code is the same as in the
transmitter
RF24 radio(9, 10); //CSN and CE pins

// The sizeof this struct should not exceed 32 bytes
struct Received_data {
  byte ch1;
  byte ch2;
  byte ch3;
  byte ch4;
  byte ch5;
  byte ch6;
  byte ch7;
};

Received_data received_data;

Servo channel_1;
Servo channel_2;
Servo channel_3;
Servo channel_4;
Servo channel_5;
Servo channel_6;
Servo channel_7;

int ch1_value = 0;
int ch2_value = 0;
int ch3_value = 0;
int ch4_value = 0;
int ch5_value = 0;
int ch6_value = 0;
int ch7_value = 0;

```

```

void reset_the_Data()
{
    // 'safe' values to use when NO radio input is detected
    received_data.ch1 = 0;          //Throttle (channel 1) to 0
    received_data.ch2 = 127;
    received_data.ch3 = 127;
    received_data.ch4 = 127;
    received_data.ch5 = 0;
    received_data.ch6 = 0;
    received_data.ch7 = 0;
}

/*****/

void setup()
{
    //Attach the servo signal on pins from D2 to D8
    channel_1.attach(2);
    channel_2.attach(3);
    channel_3.attach(4);
    channel_4.attach(5);
    channel_5.attach(6);
    channel_6.attach(7);
    channel_7.attach(8);

    //We reset the received values
    reset_the_Data();

    //Once again, begin and radio configuration
    radio.begin();
    radio.setAutoAck(false);
    radio.setDataRate(RF24_250KBPS);
    radio.openReadingPipe(1,pipeIn);

    //We start the radio communication
    radio.startListening();
}

/*****/

unsigned Long lastRecvTime = 0;

//We create the function that will read the data each certain time
void receive_the_data()
{
    while ( radio.available() ) {
        radio.read(&received_data, sizeof(Received_data));
        lastRecvTime = millis(); //Here we receive the data
    }
}

/*****/

void Loop()
{
    //Receive the radio data
    receive_the_data();

    ///////////////This small if will reset the data if signal is lost for 1 sec.
    ////////////////////////////////////////
    unsigned long now = millis();
    if ( now - lastRecvTime > 1000 ) {
        // signal lost?
        reset_the_Data();
        //Go up and change the initial values if you want depending on
        //your applications. Put 0 for throttle in case of drones so it won't
        //fly away
    }

    ch1_value = map(received_data.ch1,0,255,1000,2000);

```

```

ch2_value = map(received_data.ch2,0,255,1000,2000);
ch3_value = map(received_data.ch3,0,255,1000,2000);
ch4_value = map(received_data.ch4,0,255,1000,2000);
ch5_value = map(received_data.ch5,0,1,1000,2000);
ch6_value = map(received_data.ch6,0,1,1000,2000);
ch7_value = map(received_data.ch7,0,255,1000,2000);

```

```

//Create the PWM signals
channel_1.writeMicroseconds(ch1_value);
channel_2.writeMicroseconds(ch2_value);
channel_3.writeMicroseconds(ch3_value);
channel_4.writeMicroseconds(ch4_value);
channel_5.writeMicroseconds(ch5_value);
channel_6.writeMicroseconds(ch6_value);
channel_7.writeMicroseconds(ch7_value);

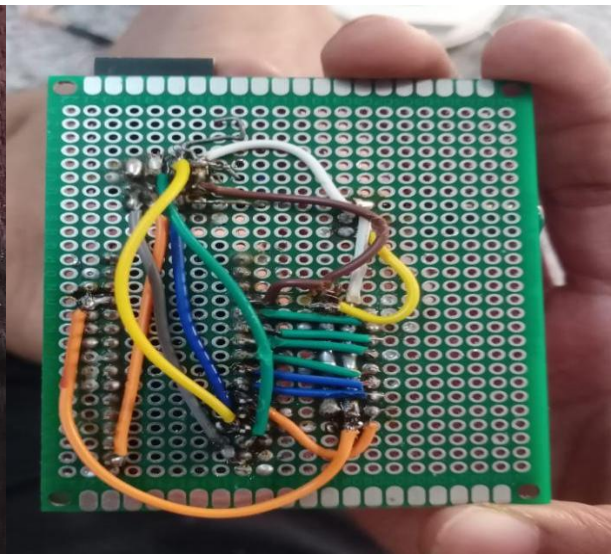
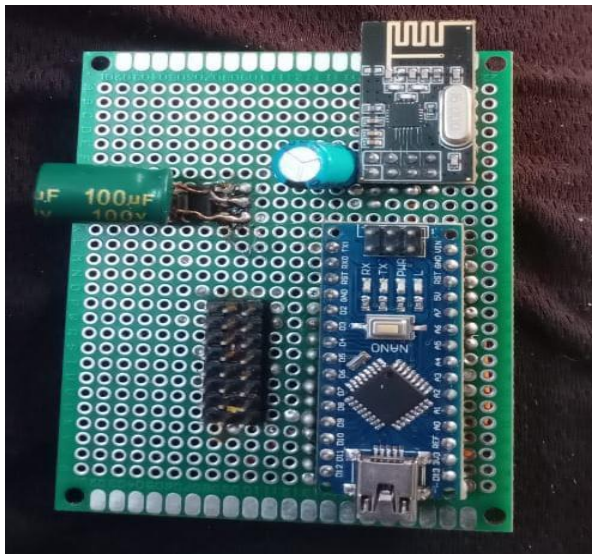
```

```

} //Loop end

```

PICTURES



The above pictures are of the receiver module .



This is the picture of the transmitter alongside the receiver



This is the aircraft for which I was building the transmitter and receiver (4 feet in length , 1000cm wingspan)

Working Video File :



Working_Vid.mp4

Credits & References

This project was inspired in part by educational content from Electronoobs.

Electronoobs provides clear and practical explanations related to embedded systems, electronics, and RC projects, which helped inform the initial conceptual understanding of transmitter–receiver systems.

- Website: <https://electronoobs.com/>
- YouTube Channel: <https://www.youtube.com/@ELECTRONOOPS>

All hardware selection, firmware implementation, debugging, testing, and system integration presented in this repository were independently developed and adapted for this project.

GitHub Repository link: <https://github.com/fuzzytron-Vivek/diy-tx-rx>